Is the model any good?

- **Every test score is a claim:** this is how the model will do on data it has not seen
- **The claim is easy to get wrong on environmental data,** and the mistake makes the model look better than it is
- **Three ideas carry the week:** a baseline, a split that matches the application, and settings chosen on data the score never sees
- **Then one more question:** not whether the model is good, but how wrong a particular prediction might be

A baseline

climatology, persistence, or the mean. No score means anything on its own

↓

A split that matches the application

many splits, not one, and folds that respect space and time

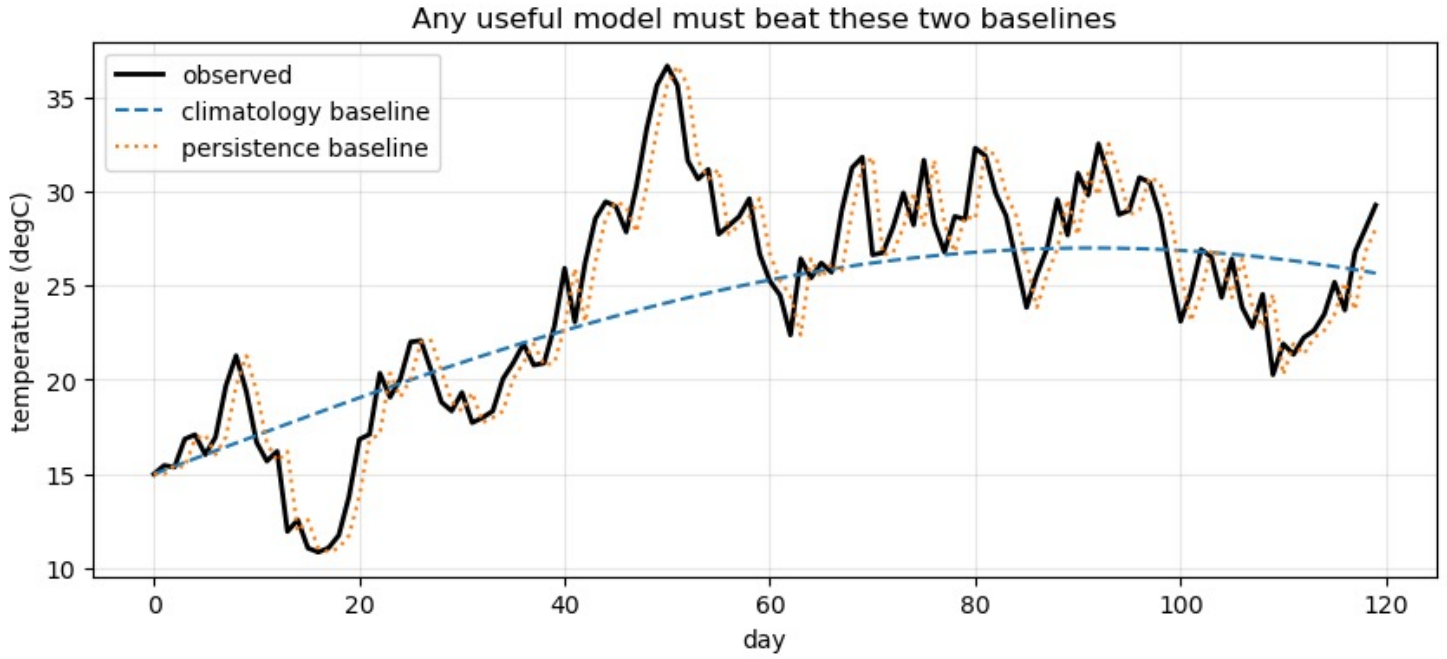
↓

Settings chosen without cheating

validation data or cross-validation chooses them; the test set is scored once

Following the Week 6 lecture notes and Géron, Hands-On Machine Learning, 3rd ed., Ch. 2

Always compare against a baseline

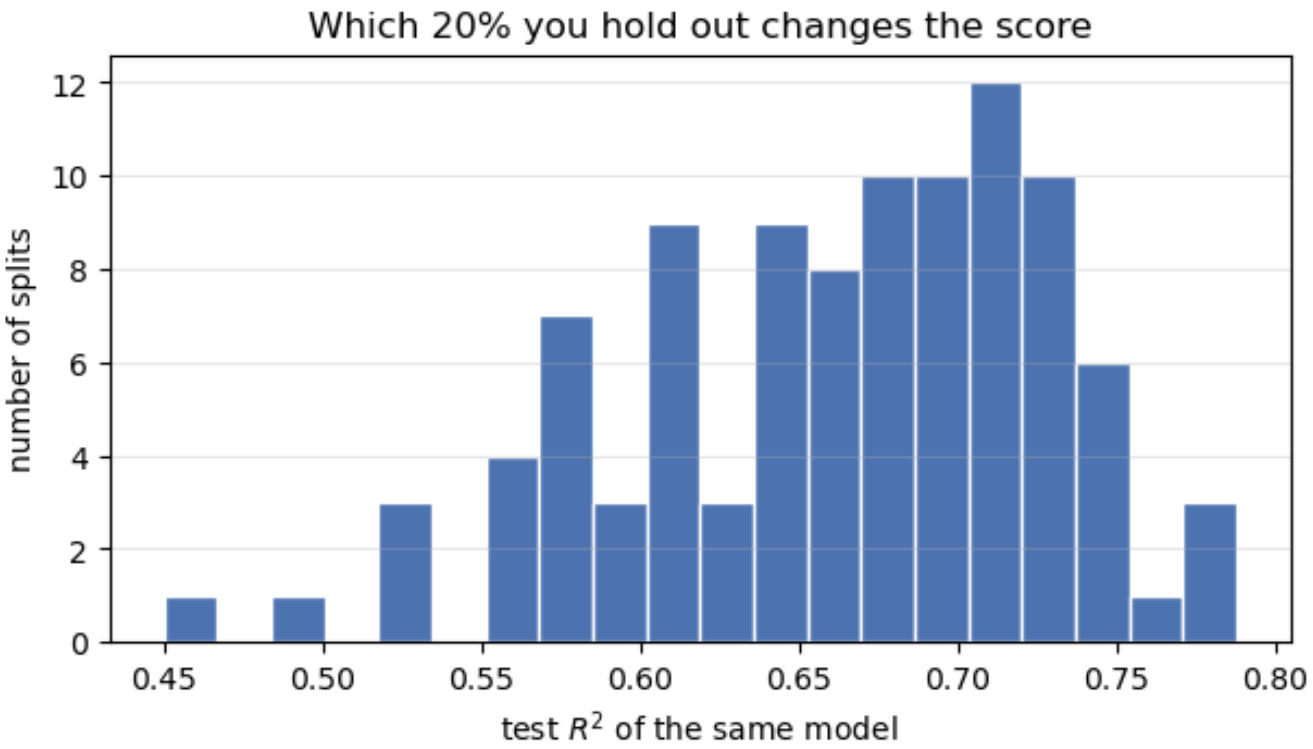


Baseline	R^2
Climatology: the seasonal mean	0.87
Persistence: yesterday's value	0.96

- **Climatology:** the long-term mean for that place and time of year
- **Persistence:** the most recent observation
- **Both are hard to beat,** because weather carries over from day to day. That is the point
- **A skill score without its baseline is not a result**

Figure from the Week 6 lecture notes

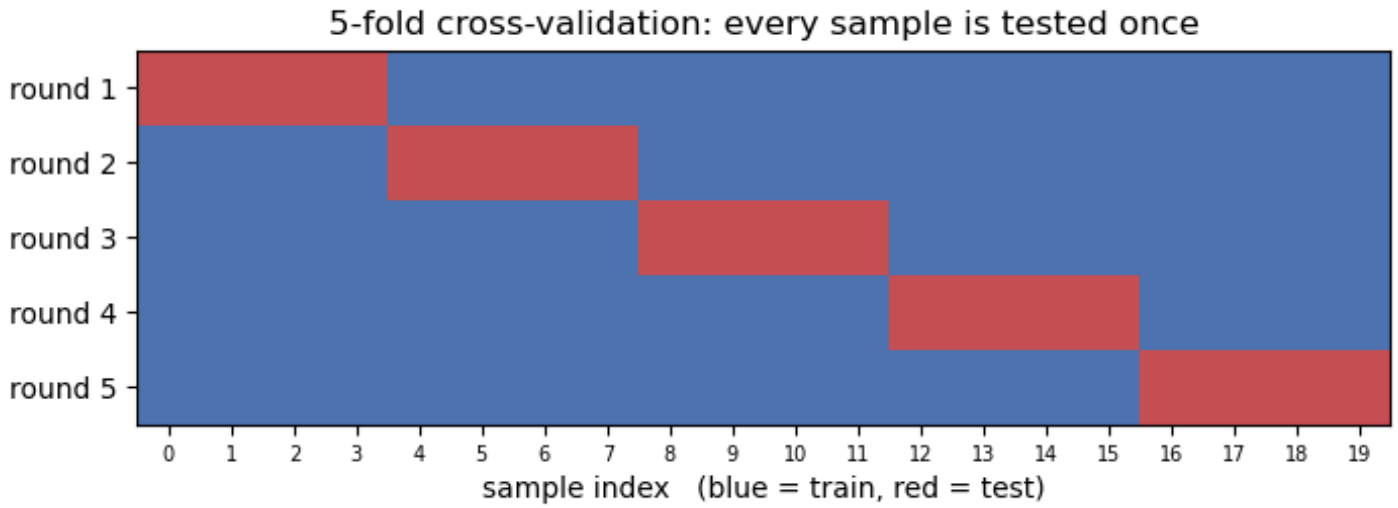
One hold-out score is one noisy number



- **Same model, same 300 samples, 100 random 80/20 splits**
- **Test R^2 runs from 0.45 to 0.79.** Which 60 samples you held out decided the score
- **A single hold-out is one draw from this histogram,** and you do not know where in it you landed
- **Small data sets make it worse,** and station records, field campaigns, and model ensembles are small

Figure from the Week 6 lecture notes

K-fold cross-validation



What it buys: all the data tested, most of the data trained on in every round, and a spread.
What it costs: k fits instead of one.

- **Cut the data into k folds.** Hold out each in turn, fit on the rest, score on the held-out fold
- **Every sample is tested once** and trained on k − 1 times
- **Report the mean and the spread** of the k scores, not one number
- **k = 5 or 10 is standard.** Leave-one-out costs n fits for little gain

Figure from the Week 6 lecture notes

K-fold in scikit-learn: the loop and the shortcuts

```
kf = KFold(n_splits=5, shuffle=True, random_state=0)

scores = []
for train_idx, test_idx in kf.split(X):
    model.fit(X[train_idx], y[train_idx])
    scores.append(model.score(X[test_idx], y[test_idx]))

print(np.mean(scores), np.std(scores))
```

```
scores = cross_val_score(model, X, y, cv=kf, scoring="r2")

detail = cross_validate(model, X, y, cv=kf,
    scoring=["r2", "neg_root_mean_squared_error"],
return_train_score=True)
```

```
rkf = RepeatedKFold(n_splits=5, n_repeats=20, random_state=0)
# 100 fits
scores = cross_val_score(model, X, y, cv=rkf, scoring="r2")
```

Depth-4 tree, 300 samples	R^2
Five fold scores	0.69, 0.67, 0.58, 0.54, 0.79
5-fold mean ± spread	0.65 ± 0.09
Repeated 20 times (100 fits)	0.67 ± 0.07
100 single hold-outs	0.66 ± 0.07

The splitter is the thing to understand. KFold yields a (train rows, test rows) pair per round. Everything else is bookkeeping.

Error metrics come back negated, because scikit-learn maximizes every score. Flip the sign.

Repeat when the fits are cheap. Five scores give a rough spread; a hundred give a reliable one.

Following the Week 6 lecture notes and Géron, Hands-On Machine Learning, 3rd ed., Ch. 2

The rules that come up every time

Rule	Why
Shuffle before cutting folds, unless order matters	KFold without shuffle takes contiguous blocks; rows are usually sorted by something
Stratify for classification (StratifiedKFold)	keeps class proportions equal across folds; essential when a class is rare
Put preprocessing inside a Pipeline	a scaler, imputer, or encoder fitted on all the data has seen every test fold
Report mean and spread, not mean and standard error	fold scores share training data and are not independent
Refit on all the data at the end	the k cross-validation models are throwaway; the deployed model is one fit on everything
Flip the sign on error metrics	scikit-learn maximizes, so RMSE arrives as neg_root_mean_squared_error

Every one of these is a form of the same rule: nothing about the test fold may touch the model before it is scored.

Following the Week 6 lecture notes and Géron, Hands-On Machine Learning, 3rd ed., Ch. 2

The assumption underneath: independent samples

- **Random k-fold assumes samples are independent and identically distributed:** knowing one tells you nothing about another
- **Geophysical data violate this almost everywhere**
- **A random split then puts near-copies of each test sample into the training set.** The model scores by lookup, and the score measures how smooth the field is
- **The inflation is silent.** Nothing in the code warns you

In space

adjacent grid cells and nearby stations are near-duplicates

In time

today looks like yesterday; a year of daily rows is far fewer than 365 independent samples

Across the sample

one satellite orbit, one instrument, one model ensemble: shared structure

Following the Week 6 lecture notes and Géron, Hands-On Machine Learning, 3rd ed., Ch. 2

A random split can misrepresent the data: stratify

Income category	Overall %	Stratified %	Random %	Stratified error	Random error
1 (below $15k)	3.98	4.00	4.24	+0.4 %	+6.4 %
2	31.88	31.88	30.74	0.0 %	−3.6 %
3	35.06	35.05	34.52	0.0 %	−1.5 %
4	17.63	17.64	18.41	+0.0 %	+4.4 %
5 (above $60k)	11.44	11.43	12.09	−0.1 %	+5.6 %

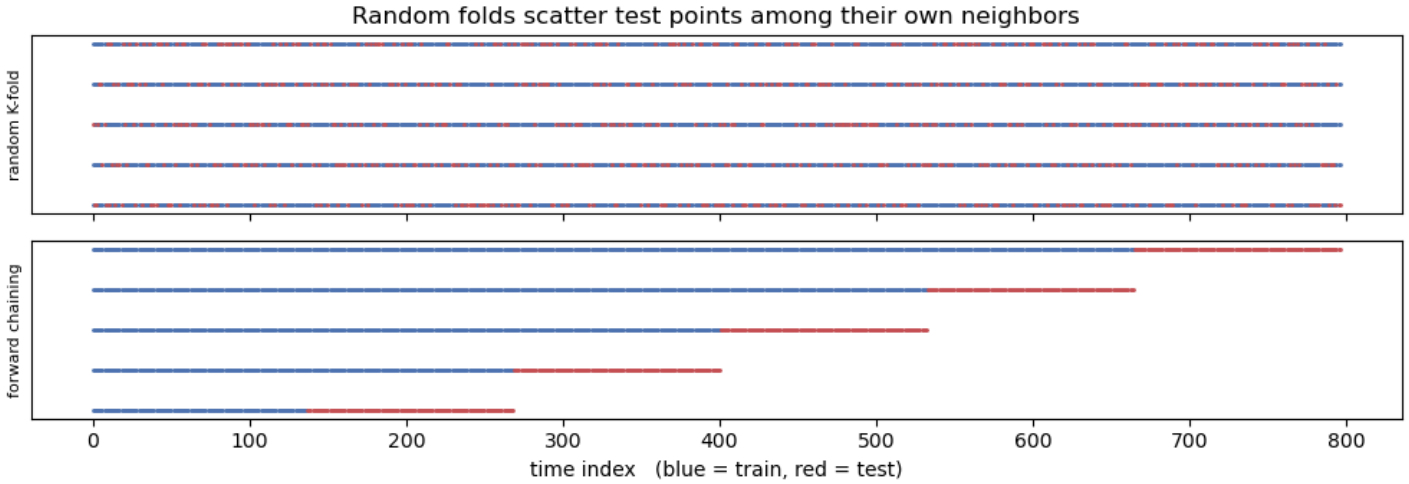
```
train_set, test_set = train_test_split(housing, test_size=0.2,
    stratify=housing["income_cat"], random_state=42)
```

Reading the table. Same data, same 20 percent. The random test set has six percent too many of the poorest and the richest districts, by chance. The stratified one matches the population in every category. Neither model has been fitted yet, and the score is already biased one way or the other.

- **Sampling bias:** a random test set can be unrepresentative by chance, and a small data set makes it likely
- **Stratified sampling:** divide into strata, draw the right number from each. The survey company does not call people at random
- **One argument in scikit-learn,** and StratifiedKFold does the same inside cross-validation. Classifiers get it by default
- **Our strata:** a rare class, a station, a river. Every fold should contain every gauge
- **What it does not fix:** dependence. Stratified daily data are still autocorrelated daily data

Géron, Hands-On Machine Learning, 3rd ed., Ch. 2, Create a Test Set. Table recomputed from the California housing data with Géron's five income categories

Time: random folds train on the future

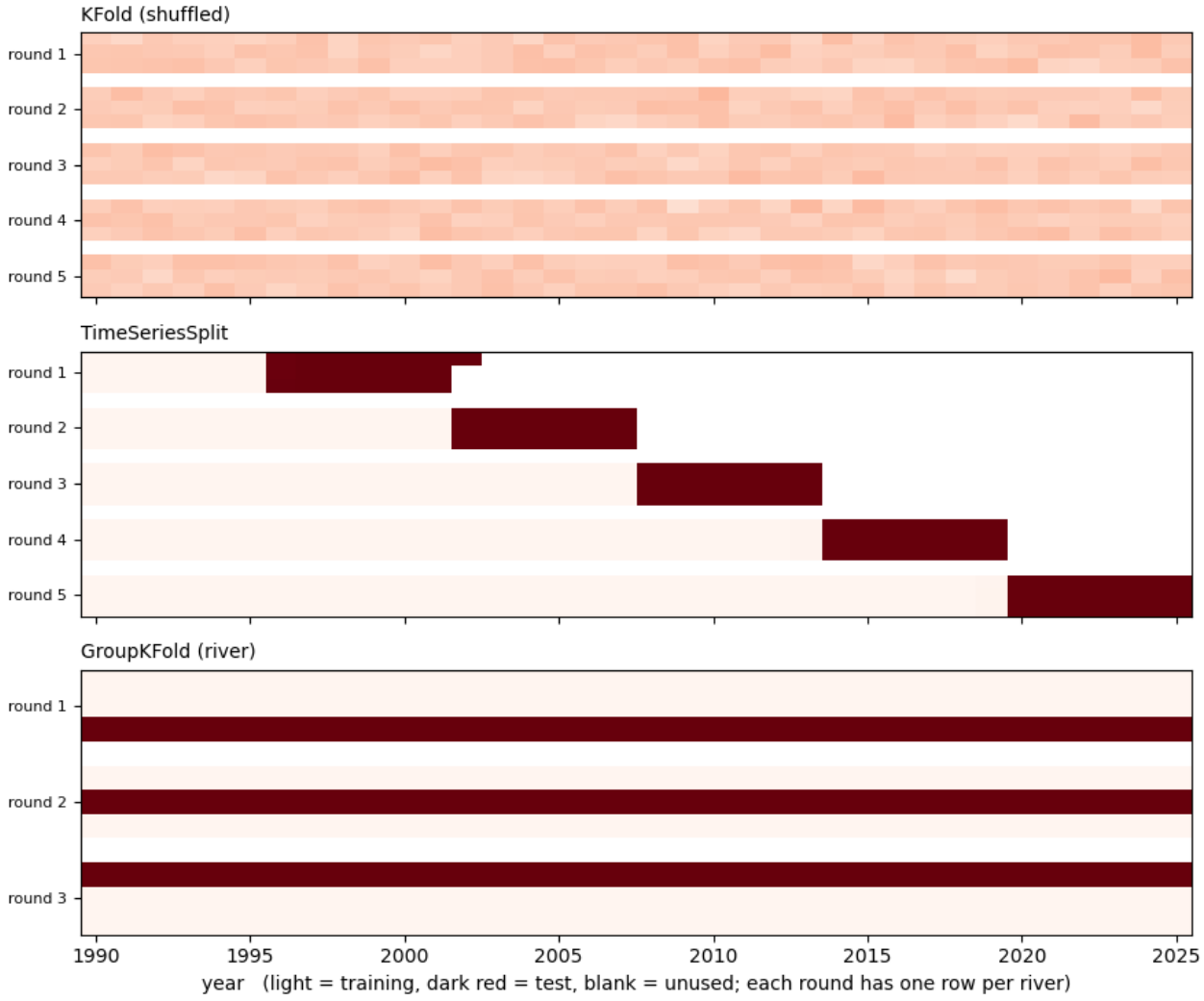


Same forest, same data	R^2
Random 5-fold	0.44
Forward chaining (TimeSeriesSplit)	−0.36
Persistence	0.94

- **A slow unobserved mode makes the target unlearnable from the features**
- **Random folds say 0.44:** a model that seems to work
- **Forward chaining says worse than the mean.** That is the truth, and persistence beats everything
- **The random score was not inflated, it was invented,** from the model recognizing its neighbors in time

Figure from the Week 6 lecture notes

Three splitters, three questions

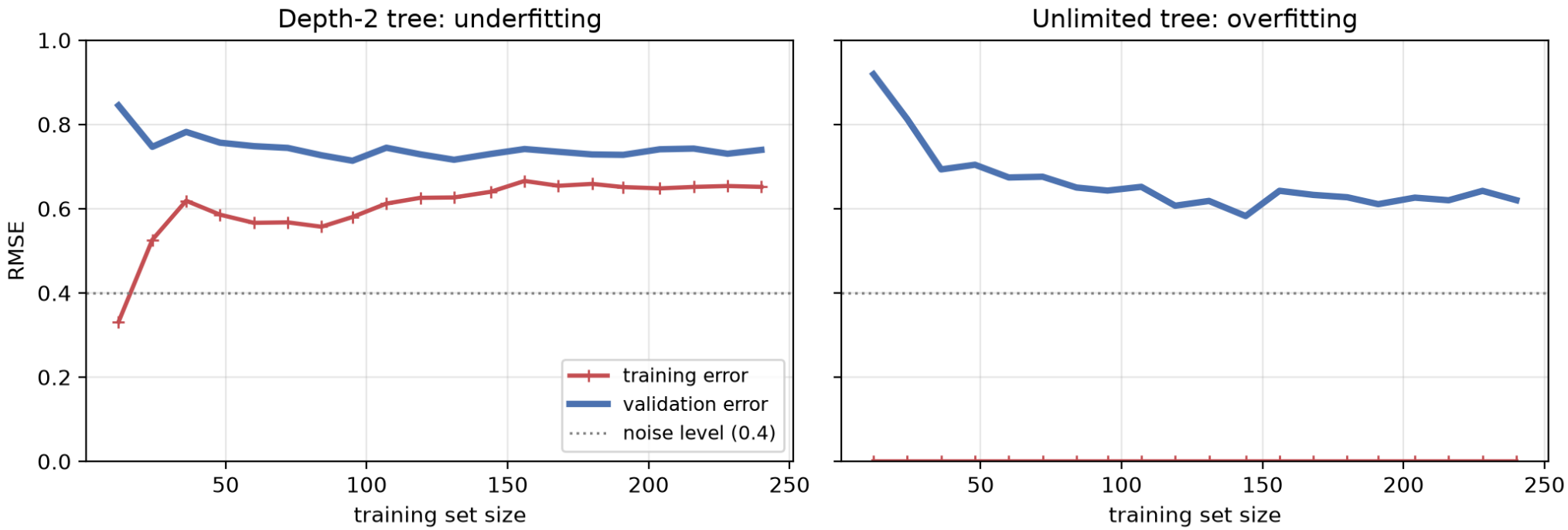


Split, depth-5 tree	RMSE (log flow)	Persistence
A: random 80/20	0.443	0.447
B: before and after 2018	0.414	0.433
C: hold out the Delaware	0.663	0.606
C: hold out the Willamette	0.761	0.409
C: hold out the Yellowstone	0.648	0.256

- **Each splitter asks a different question:** fill in a missing week, forecast next year, or handle a river never seen
- **Splits A and B agree and sit near persistence.** Split C fails on every river
- **Same data, same code, same tree.** The split is the question

Week 6 tutorial: USGS daily streamflow at the Delaware, Willamette, and Yellowstone gauges, 1990 to 2025

Learning curves: too simple, or too flexible?

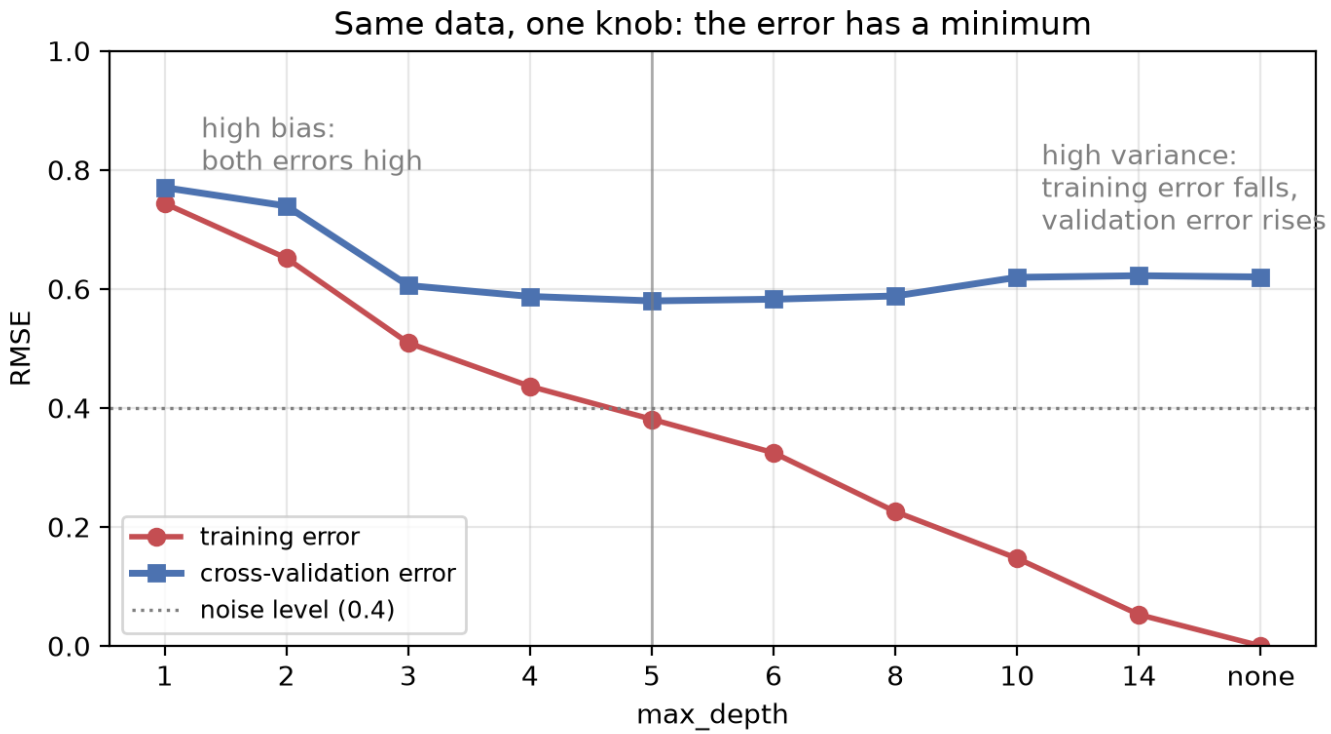


Underfitting, left: both curves plateau high and close together. More data will not help. Use a more flexible model or better features.

Overfitting, right: training error near zero and a gap to the validation curve. More data narrows the gap. So does a simpler model.

Géron, Hands-On Machine Learning, 3rd ed., Ch. 4, Learning Curves, with learning_curve(). Data: the synthetic set from the Week 6 lecture notes, noise standard deviation 0.4

The bias/variance trade-off



Reading the figure. Depths 1 to 3 are the bias regime, depth 8 and beyond the variance regime. The minimum at depth 5 is still well above the noise level of 0.4, because the irreducible part stays.

Bias

wrong assumptions: a model too simple for the data. High bias means underfitting

Variance

sensitivity to the particular training sample: many degrees of freedom fit the noise. High variance means overfitting

Irreducible error

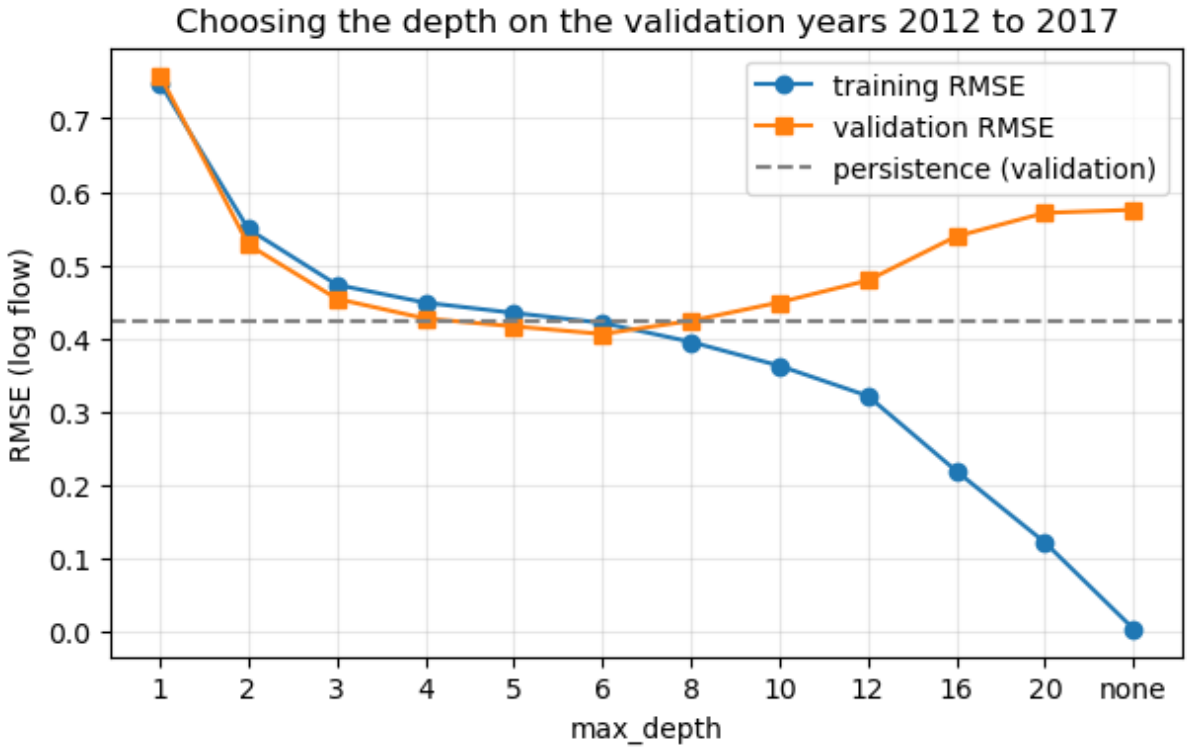
noise in the data itself. Only better data reduces it: fix the sensor, drop the bad retrievals

More complexity: variance up, bias down. Less: the reverse. The validation minimum is where the sum is smallest, and it never drops below the noise.

Géron, Hands-On Machine Learning, 3rd ed., Ch. 4, The Bias/Variance Trade-off. Validation curve on the synthetic data from the Week 6 lecture notes

Choosing hyperparameters without cheating

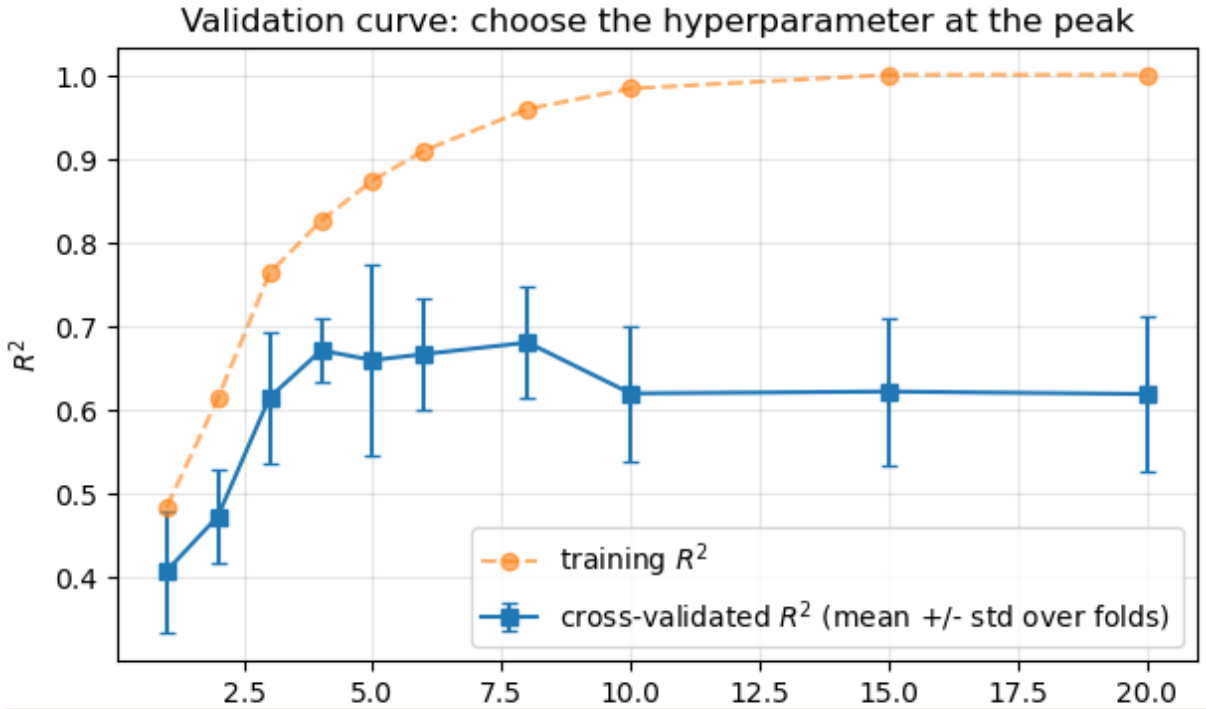
Training	Validation	Test
before 2012	2012 to 2017: chooses the depth	2018 onward: scored once



- **A hyperparameter is a setting fit does not learn:** depth, leaf size, penalty strength, number of trees
- **The rule:** the data that choose it cannot be the data that score the model. Picking the best test score picks a lucky setting
- **The three-way split:** train, validate to choose, test once at the end
- **The validation curve bottoms out at depth 6.** Refit on training plus validation, then one test score: 0.410 against persistence at 0.433

Week 6 tutorial: USGS daily streamflow at the Delaware, Willamette, and Yellowstone gauges, 1990 to 2025

Tuning by cross-validation: GridSearchCV




```
grid = {"max_depth": [1, 2, 3, 4, 5, 6, 8, 10, 15, 20]}
search = GridSearchCV(DecisionTreeRegressor(), grid,
                      cv=KFold(5, shuffle=True), scoring="r2")
search.fit(X_train, y_train)          # 10 depths x 5 folds
search.best_params_, search.best_score_, search.score(X_test, y_test)
```

- **For each candidate, k-fold on the training set;** keep the best mean, refit, test once
- **The curve plateaus between depths 4 and 8.** The error bars say the choice inside the plateau is noise
- **RandomizedSearchCV** samples combinations; searching depth and leaf size together beat the depth grid
- **best_score_ is optimistic:** the maximum of noisy numbers. Nested CV gave 0.66 against a best_score_ of 0.68

Figure from the Week 6 lecture notes; GridSearchCV after Géron, Ch. 2

After the search: inspect the winner, test once

Randomized search, 20 candidates	max_depth	min_samples_leaf	CV R^2	± std
rank 1	20	6	0.713	0.046
rank 2	14	9	0.697	0.055
rank 3	8	10	0.692	0.052
rank 4	13	2	0.685	0.031
rank 20	19	36	0.405	0.079

Tutorial final tree, test years, RMSE by category	Model	Persistence
Delaware	0.559	0.597
Willamette	0.369	0.379
Yellowstone	0.235	0.249
December to February	0.461	0.490
March to May	0.449	0.475
June to August	0.349	0.373

- **Read cv_results_, not only best_params_.** The top five candidates sit within one standard deviation: a plateau, not a point
- **Inspect best_estimator_.** It is already refit on all the training data. Feature importances, and the errors by category: by river, by season, not only the mean
- **The tutorial tree beats persistence everywhere,** by a little, and is weakest in winter and spring on every river. The score alone would not have said so
- **Then one test score, and stop.** Géron: do not tweak after measuring the generalization error, or you are overfitting the test set

Géron, Hands-On Machine Learning, 3rd ed., Ch. 2, Randomized Search, Analyzing the Best Models and Their Errors, Evaluate Your System on the Test Set. Top: Week 6 notes. Bottom: Week 6 tutorial, USGS streamflow

Leakage: when the test set reaches the model

Through preprocessing

A scaler, imputer, encoder, or feature selector fitted on all the data has seen the test set. Put every step in a Pipeline so it is refitted inside each training fold.

Tutorial, ridge regression: scaler on all rows 0.4186, inside the pipeline 0.4186. Invisible here, large for an imputer or encoder, silent either way.

Through a feature

A centered 15-day smoothing window reaches seven days into the future, to the very day being predicted.

Tutorial: legitimate tree 0.410, with the centered feature 0.319, with a trailing window 0.404. Nothing in the code looks wrong.

- **The question to ask of every feature:** would I have this number at the time the prediction is needed?
- **The usual culprits:** centered windows, shift in the wrong direction, future-dated covariates, a column that is the label in disguise
- **If a result seems too good, it probably is.** In this field an unexpectedly high score is more often leakage than a breakthrough

Week 6 tutorial: USGS daily streamflow at the Delaware, Willamette, and Yellowstone gauges, 1990 to 2025

The test score has error bars too

```
from scipy import stats
sq = (final_predictions - y_test) ** 2
np.sqrt(stats.t.interval(0.95, len(sq) - 1,
                         loc=sq.mean(), scale=stats.sem(sq)))
```

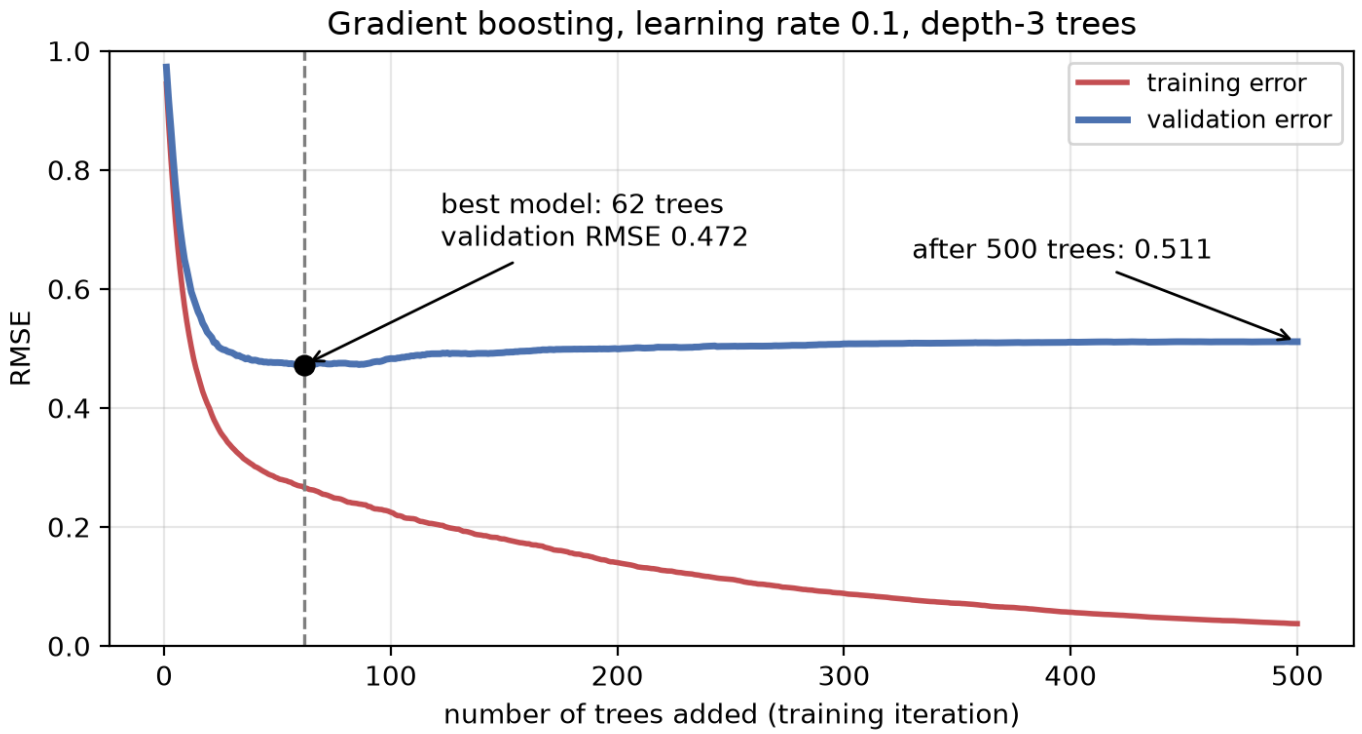
Model	Test cases	Test RMSE	95% interval
Géron's housing forest	4,128 districts	$41,424	$39,275 to $43,467
Tutorial tree, depth 6, 2018 onward	8,745 river-days	0.410	0.399 to 0.420
Persistence, same days	8,745 river-days	0.433	0.421 to 0.444

Reading the table. The two tutorial intervals do not overlap. The tree's edge over persistence is small, and it is real. Without the intervals that sentence could not be written.

- **A test score is an estimate from a finite sample,** so it has a standard error. Géron: what if the new model is only 0.1 percent better?
- **Three lines:** a t-interval on the squared errors, then the square root. Width shrinks with the root of the test size
- **For our data the interval is too narrow:** daily errors are autocorrelated. Score in blocks as a check. The tutorial's years run 0.40 to 0.42
- **After heavy tuning, expect the test score a little below the CV score.** Report it. Do not tune again

Géron, Hands-On Machine Learning, 3rd ed., Ch. 2, Evaluate Your System on the Test Set. Tutorial rows: Week 6 tutorial, USGS daily streamflow, test years 2018 onward

Early stopping: the validation set inside training



Reading the figure. The training error never stops falling. The validation error is lowest after 62 trees, 0.472, and is up to 0.511 by 500. Every tree after the 62nd made the model worse at its job.

- **Iterative learners:** gradient descent, boosting, neural networks. Validation error falls, bottoms out, then climbs as the noise gets fitted
- **Stop at the minimum,** or keep going and roll back to the best model. Hinton: a beautiful free lunch
- **Patience:** noisy curves need a wait after the minimum before stopping
- **The validation set is now inside training,** so its score is optimistic. The test set stays separate and is scored once

```
HistGradientBoostingRegressor(early_stopping=True,
    validation_fraction=0.2, n_iter_no_change=20)

keras.callbacks.EarlyStopping(patience=20,
    restore_best_weights=True)      # Weeks 9 and 10
```

Géron, Hands-On Machine Learning, 3rd ed., Ch. 4, Early Stopping. Gradient boosting on the synthetic data from the Week 6 lecture notes, 30 percent held out for validation

Géron's checklist, with this week's steps marked

1. Frame the problem and look at the big picture
2. **Get the data**
3. Explore the data to gain insights
4. Prepare the data to better expose the underlying patterns
5. **Explore many different models and shortlist the best ones**
6. **Fine-tune your models and combine them into a great solution**
7. Present your solution
8. Launch, monitor, and maintain your system

Get the data, item 11. Sample a test set, put it aside, and never look at it (no data snooping!).

Shortlist models, item 2. For each model, use N-fold cross-validation and compute the mean and standard deviation of the performance measure on the N folds.

Fine-tune, item 1. Fine-tune the hyperparameters using cross-validation. Unless there are very few hyperparameter values to explore, prefer random search over grid search.

Fine-tune, item 3. Once you are confident about your final model, measure its performance on the test set. Don't tweak your model after measuring the generalization error.

What the list assumes and ours must add: the folds must match the application. Hold out the future to forecast, the region to map, the station to transfer.

Géron, Hands-On Machine Learning, 3rd ed., Appendix A, Machine Learning Project Checklist. Items quoted from the appendix